Load-Blind Latency Models Pick the Wrong Edge Node: A Measurement Study of Game Server Placement

Ankam Charan Teja, Aranya Roy, Nikhil Kumar Reddy, Yashvardhan Deep, Akshit Singh

Abstract—Edge placement algorithms choose a node with an analytical latency model, propagation delay plus data size over bandwidth, that has no term for server load. Edge nodes are small, so a crowded edge can add more delay inside the server than its proximity saves, and because placement work is usually evaluated in simulators built on the same model, the error does not show. We measure it. On an emulated four-node edge–cloud testbed (0.5, 1, 2 and 4 cores at 5, 15, 30 and 40 ms round-trip time) running a real multiplayer game server and up to 160 protocol-level bot players, we record network, transport, server-process and in-game latency concurrently. The ICMP round trip never moves more than 0.3 ms from its configured value, while the 95th-percentile in-game latency of the smallest node rises from 15 ms at 40 players to 6.7–8.6 s at 60. We fit a closed-form M/M/1 model and a gradient-boosted tree model to two replicates of the load grid, compute each model's optimal placement for 40 to 160 players, and deploy every placement for real. The load-blind placement leaves its worst players at 18–30 s from 80 players on; both load-aware placements stay at 20–39 ms. The learned model predicts latency best (median error 0.9 ms against 1.8 ms and 5.8 ms, and 1.2 ms on a held-out third replicate) but, on these scenarios, does not place better than the fitted closed form. Run as a live controller on the same schedule of players, the load-blind model never moves a match and its worst players sit at a median of 1.06 s, queueing's at 166 ms and the learned model's at 33 ms. When one node silently loses half its CPU, in our runs only a variant that corrects itself from live measurements moves players off it.

I. Introduction

Putting a game server at the network edge shortens the path to its players. Edge nodes, however, are small: a base-station or street-cabinet server has a fraction of a data-centre machine's compute and little room to add more. When enough players are placed on one, its CPU saturates and the delay added inside the server grows without bound, while the network delay it saves is fixed.

Placement algorithms rank candidate nodes with an analytical model of latency,

$$\text{latency} = \text{propagation delay} + \frac{\text{data size}}{\text{bandwidth}}, \qquad (1)$$

which has a term for distance and a term for bandwidth and none for load. The error this causes is hard to see in the literature because placement policies are mostly evaluated numerically or in discrete-event simulators such as iFogSim [1], YAFS [2], EdgeCloudSim [3] and PureEdgeSim [4]. A recent taxonomy of microservice placement in fog computing states the gap directly: the majority of works evaluate by numerical analysis or simulation and fail to validate on real testbeds, so resource contention among services is not accurately captured [5].

This paper measures that error instead of modelling it. Our contributions:

- an emulated edge–cloud testbed running a real game server binary and protocol-level clients, with latency measured at four depths at once, so delay is attributed to a layer rather than inferred;
- a labelled dataset mapping (node, load) to measured tail latency, three replicates (one held out for testing), released with the code;
- an end-to-end comparison in which each latency model's optimal placement is deployed and measured, so models are graded by a clock rather than by each other;
- a live placement controller on the same testbed, run under the load-blind, queueing and learned models on the same schedule of players, and a test in which a node loses half its CPU mid-run without any model being told.

Terms. The round-trip time (RTT) is the time a message takes to reach a server and come back. p95 is the 95th percentile: 95% of messages are faster than it, so it describes what the unluckiest players feel rather than the average. Tick time (MSPT, milliseconds per tick) is how long the server's game loop takes for one step; the game runs 20 steps a second, so its budget is 50 ms. Utilization u is players divided by 100 per core, so $u = 1$ is a node's nominal capacity. A node's knee is the load at which its p95 leaves the flat region and climbs steeply. Median error is the prediction error that half the predictions beat. F1 grades the yes/no decision "will this node break the latency budget at this load?": 1 means every violation is caught with no false alarms, 0 means none is caught.

A. Background and related work

Service placement in fog and edge. Surveys of the service placement problem [6] and of microservice placement [5] catalogue objectives, constraints and algorithms. Placement formulations such as the integer program of Skarlat et al. [7] and the QoS-aware deployment search of Brogi and Forti [8] check latency against the links between nodes, and the mobile edge computing literature [9] models computation delay from CPU cycles and clock rate rather than from measured contention. Latency is commonly

modelled as in (1), with server-side processing either omitted or taken as a constant per request.

Simulators. iFogSim [1] and EdgeCloudSim [3] build on CloudSim; YAFS [2] on a discrete-event library and network graphs; PureEdgeSim [4] targets large heterogeneous deployments. All compute latency from configured link and processing parameters: how processing slows with load is whatever the configured model says, not something measured from the workload. They are indispensable for scale, but an algorithm evaluated in them is graded by the same assumptions it was designed with.

Closing the simulator gap. SimTune [10] uses a low-fidelity surrogate model to tune a high-fidelity simulator's parameters against a real edge–cloud platform, improving energy and response time of resource management. We take the opposite route: we keep no simulator, learn the load–latency relationship directly from measurements of an interactive workload, and evaluate on worst-player tail latency rather than mean response time.

Games and migration. Player tolerance to latency depends on the action: precise, fast-paced actions are the least tolerant [11], which is why we grade on the worst player's tail. Measurements of cloud gaming latency found much of the population too far from cloud data centres and motivated serving games from closer infrastructure [12], the setting studied here. Service migration in edge computing is often decided from the user's distance to the serving node [13]; our live controller migrates on measured and predicted server latency instead.

Learned models in resource management. Learning from measured behaviour instead of assuming a closed form is established in the data centre: Resource Central [14] predicts virtual machine behaviour from history to guide allocation, and Decima [15] learns cluster scheduling policies. We apply the same idea to a narrower question, the latency term inside edge placement, and keep the learned model small (three features) so that it can be compared directly with the closed forms.

Queueing-aware placement. Closed-form queueing terms such as $s/(1-u)$ for utilization u are the standard way to add load to a latency model. We include a fitted M/M/1 model as the strongest closed-form baseline rather than compare only against (1).

II. Methodology

The question is how far the load-blind model's prediction is from what players experience on a loaded edge node, and whether a model learned from measurements predicts and places better. We answer it by measuring a real game server under controlled load at four nodes, fitting three latency models to the measurements, and grading each model twice: by deploying the placements it chooses, and by letting it drive a live controller.

A. Testbed design

Five containers on one host are wired by containerlab: a client node and four candidate servers. Linux netem adds the network delay, split evenly between the two ends of each link, and each server's CPU is limited by a cgroup quota on cores no other container uses. Table I lists the nodes; the RTT is measured with ping, not assumed. Every server runs the same Paper 1.21.4 Minecraft server and the same pre-generated world. The trade-off is deliberate: the nearest node is the weakest.

TABLE I
Candidate nodes. RTT measured with ping at bring-up; every grid cell stayed within 0.3 ms of it.

node	CPU quota	measured RTT
edge1	0.5 core	5.2 ms
edge2	1 core	15.2 ms
edge3	2 cores	30.3 ms
cloud	4 cores	40.2 ms

B. Workload

Players are mineflayer bots that speak the real game protocol: each walks, turns, and sends one timestamped chat message per second. The chat echo gives an in-game round-trip time through the server's main game thread. Bots run at most ten per process. Every chat message is delivered to every player, so N players produce about N^2 deliveries per second; with one process for all bots this overloaded the client, and the client's own delay was recorded as server latency.

C. Measurement ladder

Four latencies are measured concurrently on the same path, under the same load: ICMP echo (network only, what (1) predicts); TCP handshake (adds the host's accept path); server-list status ping (adds the server process); and in-game RTT (adds the game thread). Tick time (MSPT), CPU and player count are sampled every 5 s from the server.

D. Measurement protocol

Each grid cell is one node at one load: 20 s settle, 15 s idle baseline, staggered joins, 90 s steady window; only the steady window is analysed. edge1 is measured at 1, 10, 20, 30, 40, 50 and 60 players, the others at 1, 20, 40, 60, 80 and 100. The grid is run three times: twice to fit on (the second in reverse node order) and a third time, hours later on a restarted testbed, held out for testing. Before collection the worlds are warmed, server heaps are prevented from swapping, and the game's day/night cycle, weather and hostile mobs are frozen: with the cycle running, a single player at night put edge1 14 s behind, so a cell's time of day was an uncontrolled load source.

E. Latency models

All three models predict p95 in-game latency for a node and a player count. Analytical: (1) with a 100 B message over 1 Gb/s. Queueing: RTT $+ s/(1-u)$, $u =$ players / (100 $\times$ cores), with s the median of delay $\times(1-u)$

over stable cells (2.4 ms). Learned: scikit-learn gradient-boosted trees (300 trees, depth 3, learning rate 0.05) on utilization, players and cores, predicting the logarithm of the server delay (p95 minus the network RTT), fitted on the first two grid replicates.

F. Placement experiment

A batch of 40, 80, 120 or 160 players in matches of ten is split across the four nodes to minimise the worst player's predicted p95. For each model the optimum is found by exhaustive search.

Every resulting placement is deployed on the testbed with all loaded nodes running at once, and measured with the same streams as the grid. No model's prediction is used to grade any model. Each placement was deployed once. With every node loaded at the same time the numbers differ from the grid's single-node cells: edge1 with 40 players measured 26 ms deployed against 14.5 ms in the grid, a difference we have not isolated (the client generates load for all nodes at once, and 40 is edge1's knee).

G. Live controller

Placement is also run as a controller on the live testbed. Every 2 s it reads every bot's latency samples, and every 5 s every server's player count, tick time and CPU; every 10 s it may move one match (ten players) if the move lowers the predicted worst-player p95 by at least 10 ms and at least 20%, or leaves it unchanged and lowers the players' mean by 3 ms, with a 60 s cooldown per match. A move stops the match on one server and starts it on another, so its players reconnect. New matches land on the nearest node with room, as a distance-based matchmaker would place them, so two runs differ only in the model that drives the controller.

H. Evaluation metrics

Predictions are scored per cell against the measured p95: median and mean absolute error, and the median ratio between prediction and measurement. For the decision "will this node break a budget at this load" we report precision, recall and F1 at 50 ms (one game tick) and 100 ms. Model differences are checked with a paired bootstrap over cells. A placement is scored by the worst player's measured p95, and for a single match by the regret against the best node. A live run is scored by the worst node's p95 at every tick: its median, its maximum and the share of the run above 100 ms, plus the number of moves and the time players spend reconnecting.

III. Implementation

Testbed. A containerlab topology file declares the client and the four servers and their links; tc netem sets each link's delay and a cgroup cpu.max quota on disjoint CPU sets caps each server. Every server runs the itzg/minecraft-server image with Paper 1.21.4. Before any measurement, topology/verify.sh checks link addresses, the netem queues, the RTT measured with ping, the CPU limits read back from the kernel, the host's power mode and the frozen game rules, and refuses to continue if any check fails.

Load and probes. bots/bot.js (Node.js, mineflayer) runs ten bots per process; each joins, walks, turns and sends a timestamped chat message every second, and logs the round trip of every echo. bots/probe.js times the TCP handshake and the server-list ping, and ping the ICMP round trip, all against the same server at the same time. controller/collect.sh runs one grid cell (settle, baseline, joins, steady window) and samples tick time, CPU and players over RCON every 5 s; grid.sh sweeps every node's ladder.

Dataset and models. dataset/build_dataset.py joins the raw streams of every cell into 30 s windows (106 cells, 318 rows). placement/models.py implements the three models behind one interface, predict(node, players), so that every algorithm runs unchanged with any of them. model/evaluate.py, metrics.py and uncertainty.py produce every score in Section IV; placement/experiment.py runs the exhaustive search and controller/deploy_plan.sh deploys each chosen placement.

Live controller. live/controller.py runs the 2 s tick loop; live/testbed.py starts and stops matches inside the client container, tails the bots' latency logs and reads each server's counters; live/policy.py holds the placement and move rules and the live-corrected model. A FastAPI server pushes the state to a browser dashboard over a WebSocket, and live/load.sh drives a session from a terminal: players join where the model chooses, a crowd can be forced onto a node, and a node's CPU can be cut and restored. Every run writes its settings, a per-tick timeline, every migration and a summary. 26 unit tests cover the decision rules, the log parsing and the live correction. A static build of the dashboard replays the recorded runs from their logs and is hosted on GitHub Pages.

Reproducibility. The repository holds the topology, every raw measurement stream of every cell (including the discarded runs and the artefact that spoiled each), the dataset builder, the models, the placement experiment, the live controller with its dashboard and unit tests, and the run logs behind every table and figure here. One command per step rebuilds each artefact from the raw data.

IV. Results

A. The network is flat; the game thread is not

Fig. 1 shows measured p95 per node against player count, with the analytical prediction drawn flat. Across the 50 cells of the two fitting replicates ICMP p95 stays within 0.3 ms of the configured RTT. netem fixes the link delay, so this mainly shows that the hosts' network stacks added nothing under load: whatever grows, grows above the network. edge1 stays within 10 ms of its 5 ms RTT up to 40 players and reaches 6.7–8.6 s at 60, where its tick time jumps from 6.5 to 46 ms. edge2 is flat to 60 players (23 ms); its knee is at 80 players (one replicate already

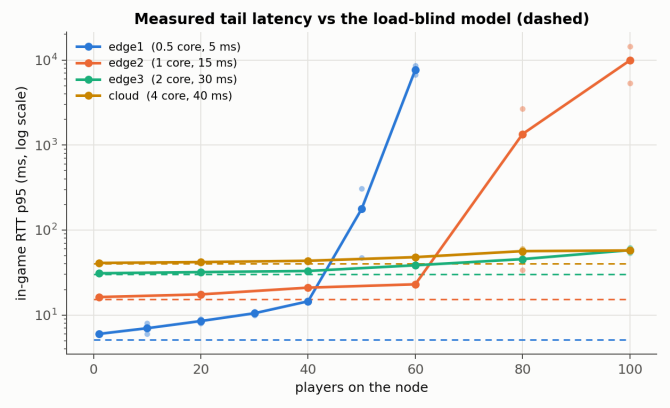


Fig. 1. Measured p95 in-game RTT per node (solid; dots are the two replicates) against the load-blind model's prediction (dashed).

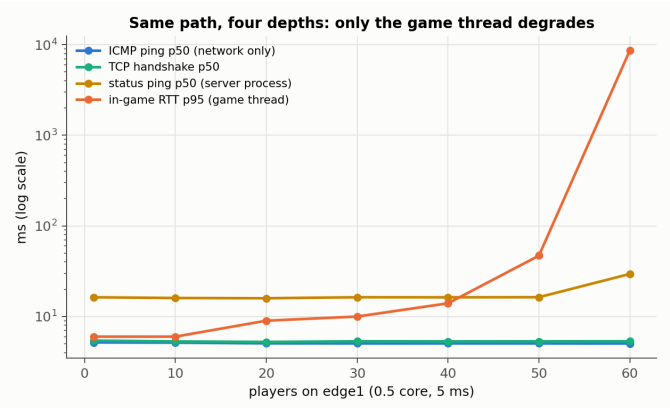


Fig. 2. The four probes on edge1 under rising load, measured concurrently on the same path. Only the probe that waits for the game thread degrades by orders of magnitude.

measured 2.7 s there, the other two 34–44 ms) and every replicate breaks by 100. edge3 and cloud never reach a knee within 100 players. The tail leaves its flat region at edge1, 50 players, while mean tick time is still 6.5 ms, an eighth of the 50 ms budget.

Fig. 2 attributes the delay. On edge1, from 1 to 60 players, the ICMP p95 stays at 5.1–5.3 ms and the TCP handshake at 5.3–5.4 ms; the server-list status ping, answered by the server process outside the game loop, moves only at 60 players (16 to 32 ms); the in-game round trip, which waits for the game thread, goes from 6 ms to 6.7–8.6 s. The network term the formula models is the one term that does not move.

Utilization alone does not place the nodes on one curve (Fig. 3). At the same u the larger nodes show more server delay: cloud at $u = 0.2$ (80 players) adds 13–20 ms, edge1 at $u = 0.2$ (10 players) 1–3 ms. Per-player cost grows with the absolute number of players (every chat message is broadcast to all of them), which a single $u = P/(100\,c)$ cannot express. This is why the learned model takes players and cores as well as utilization, and

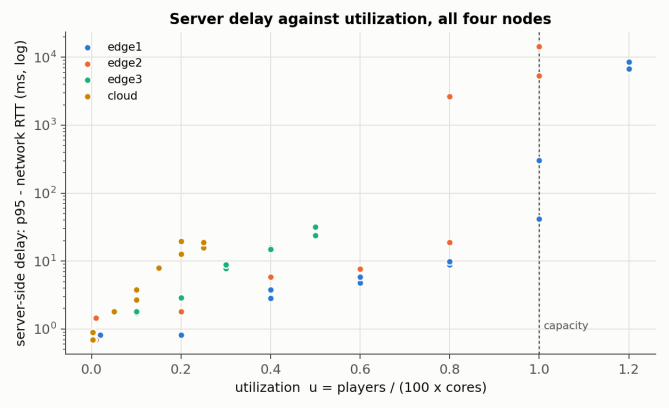


Fig. 3. Server-side delay (in-game p95 minus network RTT) against utilization, all grid cells. Nodes of different size do not share one curve.

a limit of the M/M/1 form, which is a function of u only.

B. Rank inversion

For a single match, the analytical model always picks edge1. That is correct up to 40 players (edge1 14.5 ms, edge2 21 ms) and wrong at 60, where edge1 measures 7.7 s and edge2 23 ms. The regret, the chosen node's latency minus the best node's, is 7.66 s for the worst player. Both load-aware models switch to edge2 at 60 and have zero regret at every measured load.

C. Prediction error

Table II fits each model on one replicate and predicts the other. The learned model has the lowest median and mean error; the queueing model is accurate in the flat region, but as u approaches 1 its $1/(1-u)$ term grows without bound, so near the knee it predicts enormous delays and its mean error is large. Fitted on three nodes and tested on the fourth, the learned model transfers best to edge1, edge3 and cloud, and the queueing model to edge2. On a third replicate measured hours later and never used in any fit (25 cells), the ranking holds: median error 1.2 ms learned, 2.3 ms queueing, 6.9 ms analytical. To see how stable these differences are, we resampled the cells with replacement 2000 times and recomputed every model's error on the same resample each time (a paired bootstrap). This puts the learned model's cross-replicate median error at 0.9 ms [0.3, 1.5] against queueing's 1.8 ms [1.6, 5.0], lower in every resample; on the 25 held-out cells it is lower in 95%. On those 25 held-out cells, asked whether a cell breaks a 50 ms budget, the learned model scores F1 0.92 (all six violations caught), the queueing model 0.67 and the analytical model 0, since it never predicts a violation.

D. Deployed placements

Table III and Fig. 4. The analytical model predicts 5–15 ms for every scenario and fills the nearest node to its 100-player cap before using the next; from 80 players its worst players wait 18–30 s, about 510 to 1500 times longer

TABLE II
Cross-replicate prediction error, 50 cells. Factor off: the median ratio between prediction and measurement (1.0 is exact). MAE: mean absolute error.

model	median abs. err.	factor off	MAE
analytical	5.8 ms	2.82×	768 ms
queueing	1.8 ms	1.85×	6547 ms
learned	0.9 ms	1.28×	469 ms

TABLE III
Worst node's measured p95 (ms) for each model's placement (edge1/edge2/edge3/cloud players)

players	analytical	queueing	learned
40	26 (40/0/0/0)	26 (40/0/0/0)	26 (40/0/0/0)
80	29 897 (80/0/0/0)	20 (40/40/0/0)	20 (40/40/0/0)
120	17 758 (100/20/0/0)	31 (40/80/0/0)	32 (40/70/10/0)
160	19 990 (100/60/0/0)	35 (40/80/40/0)	39 (40/70/50/0)

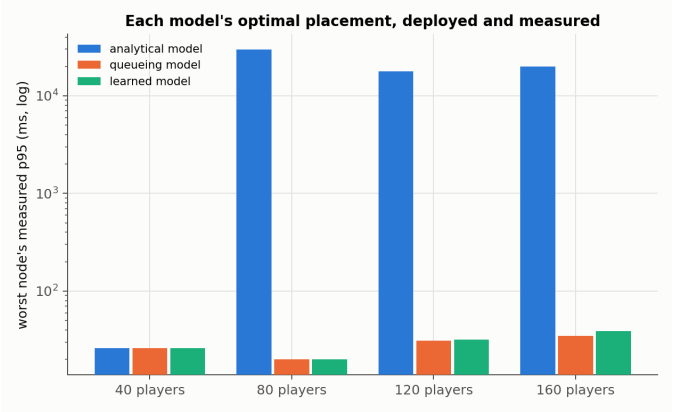


Fig. 4. Each model's optimal placement, deployed and measured (log scale).

TABLE IV
F1 for "node over 100 ms", every model scored on the same live windows

run (windows, over 100 ms)	analyt.	queue.	learned	live-corr.
analytical in control (91, 28)	0.00	0.87	0.87	0.95
queueing in control (127, 36)	0.00	0.00	0.00	0.78

than under either load-aware placement. Both load-aware models keep edge1 at 40 players and spread the rest; their placements measure within 4 ms of each other, and the learned model's predictions of its own deployments are within 12 ms. On these scenarios the learned model's advantage is in prediction, not in the placement decision: a correctly fitted closed form reaches almost the same placement.

E. Live controller

Same schedule, three controllers. A match of ten players arrives every 25 s up to 140 players, holds for 150 s, and leaves until 60 remain (13 min). Table V and Fig. 5. The analytical controller never moves a match, by construction: its prediction for a node does not change with load, so no move ever looks like an improvement. The run measures the cost of that: edge1 fills to 100 players and its p95 stays above 10 s until departures empty it. The queueing controller moves matches as often as the learned one, but its curve depends on utilization alone and rates edge2 at 80 players ($u = 0.8$) at 27 ms; edge2's knee is at 80 players, and live it sat at 1.8–3.4 s for about 200 s. The learned model predicts 65 ms there, stops edge2 at 70 and uses edge3: the effect of Fig. 3 deciding a placement. Part of the gap is chance at a knee: both runs held edge1 at exactly 40 players, which stayed backlogged in one run and recovered in the other. The learned controller keeps the worst player's p95 at a median of 33 ms; its excursions over 100 ms are the 10–25 s after each new match lands on edge1 and before it is moved away. Each move costs its ten players a reconnect: about 6 s from the stop until the last of them sends a sample from the new node, in every run.

Every model is also scored in the background on each run. A window is 10 s of unchanged load on one node, so a run yields one window per busy node every 10 s; each is graded against every model's prediction for it, made before the window was seen. Table IV gives F1 for "this node is over 100 ms". In the queueing run the windows over budget are edge1 backlogged at 40 players and edge2 at 80: loads every offline curve calls healthy. Only the model that corrects itself from the measurements catches them.

A node gets slower and no model is told. With eight matches settled, edge1's CPU quota is cut from 0.5 to 0.25 cores at 220 s (Fig. 6, Table VI). The offline models keep predicting edge1's 0.5-core curve: they keep 40 players there and are over 100 ms 99–100% of the time after the cut. A variant corrects the learned model while it runs: after each steady 10 s window it estimates a per-node factor c: the node with P players now behaves like the grid's node with $c \times P$ players, so $c = 2$ means each player costs twice as much as before. Its first version noticed the cut and moved two matches off, 30 and 70 s after it, but light, healthy windows below the new knee pulled c back down and it moved a match back, which then saw 3.9 s. With a five-minute memory of loads measured over budget, two runs were over 100 ms for 4% and 13% of the time after the cut. The spread is real: in the second run its own moves briefly loaded the nodes they landed on, and when the memory expired it probed edge1 again with one match, which saw 3–4 s while rejoining. Moves are not free and the controller does not model their cost. The memory was added after the first run; the runs show the mechanism, not a tuned result.

F. Discussion

When the formula is enough. Below a node's knee the analytical model is within tens of milliseconds, and the

TABLE V
Live controller, same 13-minute schedule, one run each (worst node's p95)

controller	median	max	over 100 ms	moves
analytical	1 058 ms	19.6 s	53%	0
queueing	166 ms	5.7 s	53%	11
learned	33 ms	817 ms	18%	10

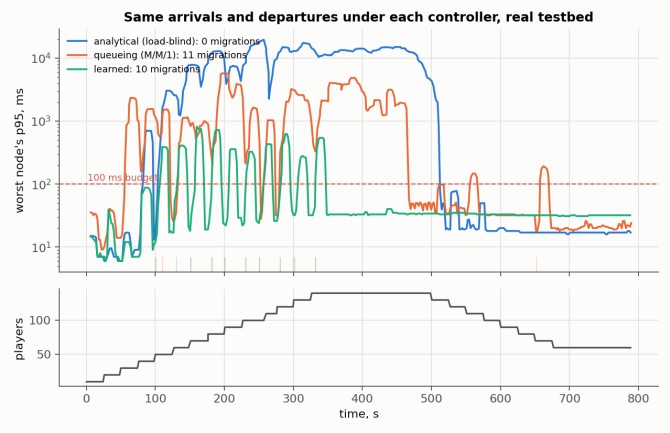


Fig. 5. Worst node's p95 over time under each controller, same arrivals and departures (bottom). Ticks mark migrations.

TABLE VI
edge1's CPU halved at 220 s: time over 100 ms

controller	runs	whole run	after the cut
queueing	1	83%	100%
learned	2	74%, 82%	99%, 99%
live, first version	1	20%	21%
live, with memory	2	2%, 14%	4%, 13%



Fig. 6. edge1's CPU halved at 220 s (dotted). Top: edge1's p95 under each controller. Middle: players each leaves on edge1. Bottom: the live model's cost factor for edge1 (true value after the cut: 2).

ranking it produces is right: every node's in-game p95 stays within about 20 ms of its network RTT. The formula fails in one regime, past the knee, and there it fails by two to three orders of magnitude (330× for one match, 510–1500× for the deployed placements), not by a margin a tolerance band could absorb. A placement algorithm evaluated only on lightly loaded scenarios, as simulator studies usually are, will never see it.

What a placement system needs. Neither load-aware model needed more than the node's size and its current player count. The deployed placements show that a closed form with one fitted parameter is enough to avoid the failure at the four offline scenarios. Live, the same form loaded edge2 past its knee because utilization alone does not capture how per-player cost grows with the number of players; in that run the learned model's more accurate curve kept it off. Accuracy matters most where a budget has to be checked near a knee, not where nodes are only ranked. The live runs add a requirement no offline model meets: capacity is not a constant. A setting on the host cut it roughly threefold in our own rig (Section V), and in our runs a CPU cut no model was told about was handled only by a controller that kept comparing its predictions with what it measured.

V. Conclusion and Future Work

The load-blind latency model used to rank edge nodes is accurate only below a node's saturation point, and above it the error is not a few milliseconds but seconds: deployed, its placements left the worst players waiting 18–30 s where a load-aware placement measured 20–39 ms on the same hardware. A model learned from measurements predicts latency best; at the four offline scenarios a fitted M/M/1 term already placed as well, but run live on the same schedule it loaded a node past its knee (median 166 ms) where the learned controller kept the worst players at a median of 33 ms and the load-blind one never moved a match. In our runs a node that silently slowed down was handled only by a model that keeps learning from measurements and remembers the loads it has seen fail. The practical lesson is that the latency term in placement must see load, and that its shape is better measured than assumed.

Limitations. One host, three replicates. All nodes share one laptop; the bootstrap intervals in Section IV cover the variation between cells, not between whole replicates, of which there are only three. At the knees the replicates differ by 6–80×, so the exact player count of a knee is not established.

Host power mode. A node capped at half a core runs in short bursts, so the CPU governor keeps its clock low unless the host is in its performance power mode. All clean runs used that mode. In the balanced mode the same players cost 2–3× the CPU (edge1 with 20 players: 42% of a core instead of 14–18%) and edge1's knee fell below

20 players. One edge1 pass and one full replicate were spoiled this way; they are excluded and kept as evidence. The same node, players and network, changed only by a setting no latency formula sees, moved the knee threefold.

Chosen, not discovered, capacities. The core counts and delays were picked to create the trade-off. The claim is that an inversion exists, is large, and is avoided by a load-aware model, not that edge nodes fail at a particular player count. The 100-players-per-core capacity the queueing model uses is untested above one core.

Synthetic workload. Bots walk and chat on a fixed schedule, and hostile mobs are disabled; real players and real mob AI add load of other shapes. The placement comparison covers four scenarios on four nodes; larger problems are where the load-aware models could differ.

Future work. more replicates on dedicated hardware, nodes pushed past their knees at higher core counts, larger placement problems, state handover instead of reconnection, and network impairments that make the bandwidth term non-trivial. Code, raw measurements and the dataset: https://github.com/ankamteja/latency-aware-edge-game-placement.

References

[1] H. Gupta, A. V. Dastjerdi, S. K. Ghosh, and R. Buyya, "iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments," Software: Practice and Experience, vol. 47, no. 9, pp. 1275–1296, 2017, arXiv:1606.02007.

[2] I. Lera, C. Guerrero, and C. Juiz, "YAFS: A simulator for IoT scenarios in fog computing," IEEE Access, vol. 7, pp. 91 745–91 758, 2019.

[3] C. Sonmez, A. Ozgovde, and C. Ersoy, "EdgeCloudSim: An environment for performance evaluation of edge computing systems," Transactions on Emerging Telecommunications Technologies, vol. 29, no. 11, p. e3493, 2018.

[4] C. Mechalikh, H. Taktak, and F. Moussa, "PureEdgeSim: A simulation framework for performance evaluation of cloud, edge and mist computing environments," Computer Science and Information Systems, vol. 18, no. 1, pp. 43–66, 2021.

[5] S. Pallewatta, V. Kostakos, and R. Buyya, "Placement of microservices-based IoT applications in fog computing: A taxonomy and future directions," ACM Computing Surveys, vol. 55, no. 14, pp. 321:1–321:43, 2023, arXiv:2207.05399.

[6] F. Ait Salaht, F. Desprez, and A. Lebre, "An overview of service placement problem in fog and edge computing," Inria, Tech. Rep. RR-9295, 2019, hAL hal-02313711.

[7] O. Skarlat, M. Nardelli, S. Schulte, M. Borkowski, and P. Leitner, "Optimized IoT service placement in the fog," Service Oriented Computing and Applications, vol. 11, no. 4, pp. 427–443, 2017.

[8] A. Brogi and S. Forti, "QoS-aware deployment of IoT applications through the fog," IEEE Internet of Things Journal, vol. 4, no. 5, pp. 1185–1192, 2017.

[9] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," IEEE Communications Surveys & Tutorials, vol. 19, no. 4, pp. 2322–2358, 2017.

[10] S. Tuli, G. Casale, and N. R. Jennings, "SimTune: bridging the simulator reality gap for resource management in edge-cloud computing," Scientific Reports, vol. 12, 2022.

[11] M. Claypool and K. Claypool, "Latency and player actions in online games," Communications of the ACM, vol. 49, no. 11, pp. 40–45, 2006.

[12] S. Choy, B. Wong, G. Simon, and C. Rosenberg, "The brewing storm in cloud gaming: A measurement study on cloud to end-user latency," in Proc. 11th Annual Workshop on Network and Systems Support for Games (NetGames), 2012, pp. 1–6.

[13] S. Wang, R. Urgaonkar, M. Zafer, T. He, K. Chan, and K. K. Leung, "Dynamic service migration in mobile edge computing based on Markov decision process," IEEE/ACM Transactions on Networking, vol. 27, no. 3, pp. 1272–1288, 2019.

[14] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms," in Proc. 26th ACM Symposium on Operating Systems Principles (SOSP), 2017, pp. 153–167.

[15] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in Proc. ACM SIGCOMM, 2019, pp. 270–288.